

Index

Sr. No.	Name of the experiment	Date	Remark	Sign
Practical Assignments on Design & Analysis of Algorithms				
1	Write programs in C to sort a list of n numbers in ascending order using selection sort, insertion sort, heap sort, radix sort. Determine the time required to sort and compare on the basis of time complexity for different values of n.	08/01/2026		
2	Write a program in C to sort a given set of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.	08/01/2026		
3	Write a program in C to implement a Merge Sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.	08/01/2026		
4	Write a program in C to implement Strassen's Matrix multiplication.	15/01/2026		
5	Write a program in C to find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	15/01/2026		
6	Write a program in C to find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm	15/01/2026		
7	Write a program in C to from a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm	05/02/2026		
8	Write a program in C to implement Knapsack problems using Greedy method.	05/02/2026		
9	Write a program in C to implement optimal binary search tree and also calculate the best case and worst case complexity.	05/02/2026		
10	Write a program in C to implement huffman Code using greedy methods and also calculate the best case and worst case	06/02/2026		

	complexity.			
11	Write a program in C to find Minimum number of multiplications in Matrix Chain Multiplication/	06/02/2026		
12	Write a Program in C to find only length of Longest Common Subsequence.	06/02/2026		
13	Write programs in C to implement DFS and BFS. Compare the time complexity.	14/02/2026		
14	Write a program in C for finding Topological sorting for Directed Acyclic Graph (DAG).	14/02/2026		
15	Write a program in C to determine if a given graph is a Hamiltonian cycle or not.	26/02/2026		
16	Write a C Program in C to implement Traveling Salesman Problem using nearest neighbor algorithm.	26/02/2026		
17	Write a program in C a to implement Graph Coloring Algorithm.	28/02/2026		
18	Write a program in C to implement Sum of Subset by Backtracking.	28/02/2026		
19	Write a program in C to solve N Queens Problem using Backtracking.	04/03/2026		
20	Write a program in C to solve 4 Queens Problem using Backtracking	04/03/2026		
21	Write a program in C to show board configuration of 4 queens problem	04/03/2026		
22	Write a program in C to find out longest common subsequence from the given strings	05/03/2026		
23	Write a program in C to find out live node, E node and dead node from a given graph.	05/03/2026		
24	Write a program in C to find out solution for travelling salesman problem using LCBB from a given matrix.	12/03/2026		
25	Write a program in C to find out solution for 0/1 knapsack problem	12/03/2026		

Vidya Pratishthan's
Arts, Science and Commerce College
Vidyanagari, Baramati
Department of Computer Science

Name:

Performed

Submitted

**Remark &
Signature**

Class: F.Y. M.Sc. (Comp. Sci.)

Roll No:

Batch No:

Experiment No: 01

Lab No:

Aim : Write programs in C to sort a list of n numbers in ascending order using selection sort, insertion sort, heap sort, radix sort. Determine the time required to sort and compare on the basis of time complexity for different values of n.

Program Code :

1.Selection Sort:

```
#include <stdio.h>
#include <time.h>

int main()
{
    int n, i, j, min, temp;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    start = clock(); // start time

    for(i = 0; i < n-1; i++)
    {
        min = i;
```

```

        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    end = clock(); // end time

    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}

```

OutPut : Enter number of elements: 5

Enter elements:

12 78 34 56 11

Sorted array:

11 12 34 56 78

2.Insertion sort:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int n, i, key, j;
```

```
    clock_t start, end;
```

```
    double time_taken;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```

int a[n];

printf("Enter elements:\n");
for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

start = clock();

for(i = 1; i < n; i++)
{
    key = a[i];
    j = i - 1;

    while(j >= 0 && a[j] > key)
    {
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = key;
}

end = clock();

time_taken = (double)(end - start) / CLOCKS_PER_SEC;

printf("Sorted array:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\nTime taken: %f seconds\n", time_taken);

return 0;
}

```

Output:

```

Enter number of elements: 5
Enter elements:
64 25 12 22 11
Sorted array:
11 12 22 25 64

```

3.Heap sort:

```
#include <stdio.h>
#include <time.h>

void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    int temp;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i)
    {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

int main()
{
    int n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```

start = clock();

for(i = n/2 - 1; i >= 0; i--)
    heapify(a, n, i);

for(i = n - 1; i > 0; i--)
{
    int temp = a[0];
    a[0] = a[i];
    a[i] = temp;

    heapify(a, i, 0);
}

end = clock();

time_taken = (double)(end - start) / CLOCKS_PER_SEC;

printf("Sorted array:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\nTime taken: %f seconds\n", time_taken);

return 0;
}

```

Output:

Enter number of elements: 6

Enter elements:

12 11 13 5 6 7

Sorted array:

5 6 7 11 12 13

4.Radix sort:

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int getMax(int a[], int n)
```

```
{  
    int max = a[0];  
    for(int i = 1; i < n; i++)  
        if(a[i] > max)  
            max = a[i];  
    return max;  
}
```

```
void countSort(int a[], int n, int exp)  
{  
    int output[n];  
    int count[10] = {0};  
  
    for(int i = 0; i < n; i++)  
        count[(a[i] / exp) % 10]++;  
  
    for(int i = 1; i < 10; i++)  
        count[i] += count[i - 1];  
  
    for(int i = n - 1; i >= 0; i--)  
    {  
        output[count[(a[i] / exp) % 10] - 1] = a[i];  
        count[(a[i] / exp) % 10]--;  
    }  
}
```



```
    for(int i = 0; i < n; i++)  
        a[i] = output[i];  
}
```

```
int main()  
{  
    int n;  
    clock_t start, end;  
    double time_taken;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    int a[n];  
  
    printf("Enter elements:\n");  
    for(int i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
  
    start = clock();  
  
    int max = getMax(a, n);  
    for(int exp = 1; max/exp > 0; exp *= 10)
```

```
        countSort(a, n, exp);

    end = clock();

    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Output:

Enter number of elements: 8

Enter elements:

170 45 75 90 802 24 2 66

Sorted array:

2 24 45 66 75 90 170 802

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 02	Roll No: Lab No:	Batch No:
Aim : Write a program in C to sort a given set of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.		

Program Code:

```

/* Quick Sort with Time Calculation */

#include <stdio.h>

#include <time.h>

void quicksort(int a[], int low, int high);
int partition(int a[], int low, int high);
int main()
{
    int a[100], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d",&n);

```

```

printf("Enter elements:\n")
for(i=0;i<n;i++)
    scanf("%d",&a[i]);
start = clock(); // Start time
quicksort(a,0,n-1);
end = clock(); // End time
time_taken = ((double)(end-start))/CLOCKS_PER_SEC;
printf("\nSorted elements in ascending order:\n");
for(i=0;i<n;i++)
    printf("%d ",a[i]);
printf("\nTime required to sort = %f seconds",time_taken);
return 0;
}

void quicksort(int a[], int low, int high)
{
    int p;
    if(low < high)
    {
        p = partition(a,low,high);
        quicksort(a,low,p-1);
        quicksort(a,p+1,high);
    }
}

int partition(int a[], int low, int high)

```

```

{
int pivot, i, j, temp;

    pivot = a[low];
    i = low+1;
    j = high;
    while(i<=j)
    {
        while(a[i] <= pivot && i<=high)
            i++;
        while(a[j] > pivot)
            j--;
        if(i<j)
        {
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }

    temp=a[low];
    a[low]=a[j];
    a[j]=temp;
return j;
}

```

Output:

Enter number of elements: 5

Enter elements:

45 23 78 12 56 10

Sorted elements in ascending order:

12 23 45 56 78

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 03	Roll No: Lab No:	Batch No:
Aim : Write a program in C to implement a Merge Sort algorithm to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.		

Program Code:

```

/* Merge Sort with Time Calculation */

#include <stdio.h>

#include <time.h>

void merge(int a[], int low, int mid, int high)
{
    int i, j, k;
    int temp[1000];
    i = low;
    j = mid + 1;
    k = low;
    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])

```

```

        temp[k++] = a[i++];
    else
        temp[k++] = a[j++];
}
while(i <= mid)
    temp[k++] = a[i++];
while(j <= high)
    temp[k++] = a[j++];
for(i = low; i <= high; i++)
    a[i] = temp[i];
}

void mergesort(int a[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high) / 2;
        mergesort(a, low, mid);
        mergesort(a, mid+1, high);
        merge(a, low, mid, high);
    }
}

int main()
{

```



```

int a[1000], n, i;

clock_t start, end;

double time_taken;

printf("Enter number of elements: ");

scanf("%d",&n);

printf("Enter elements:\n");

for(i=0;i<n;i++)

    scanf("%d",&a[i]);

start = clock();

mergesort(a,0,n-1);

end = clock();

time_taken = ((double)(end-start))/CLOCKS_PER_SEC;

printf("Sorted elements are:\n");

for(i=0;i<n;i++)

    printf("%d ",a[i]);

printf("\nTime required = %f seconds",time_taken);

return 0;

}

```

Output:

Enter number of elements: 6

Enter elements:

19 34 65 23 49 10

Sorted elements are:

10 19 23 34 49 65

<i>Vidya Pratishthan's</i> Arts, Science and Commerce College <i>Vidyanagari, Baramati</i> Department of Computer Science		Performed Submitted Remark & Signature
Name:		
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 04 Aim : Write a program in C to implement Strassen's Matrix multiplication.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

int main()
{
    int A[2][2], B[2][2], C[2][2];
    int M1, M2, M3, M4, M5, M6, M7;
    int i, j;
    printf("Enter elements of matrix A (2x2):\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &A[i][j]);
    printf("Enter elements of matrix B (2x2):\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &B[i][j]);
```

```
/* Strassen's formulas */
```

```
M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
```

```
M2 = (A[1][0] + A[1][1]) * B[0][0];
```

```
M3 = A[0][0] * (B[0][1] - B[1][1]);
```

```
M4 = A[1][1] * (B[1][0] - B[0][0]);
```

```
M5 = (A[0][0] + A[0][1]) * B[1][1];
```

```
M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
```

```
M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
```

```
/* Result matrix */
```

```
C[0][0] = M1 + M4 - M5 + M7;
```

```
C[0][1] = M3 + M5;
```

```
C[1][0] = M2 + M4;
```

```
C[1][1] = M1 - M2 + M3 + M6;
```

```
printf("Resultant matrix C:\n");
```

```
for(i = 0; i < 2; i++)
```

```
{
```

```
    for(j = 0; j < 2; j++)
```

```
        printf("%d ", C[i][j]);
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

Output:

Enter elements of matrix A (2x2):

2 3

5 7

Enter elements of matrix B (2x2):

6 7

8 9

Resultant matrix C:

36 41

86 98

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 05 Aim : Write a program in C to find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

int parent[10];

/* Function definitions (outside main) */

int find(int i)
{
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int a, int b)
{
    parent[b] = a;
}
```

```

}

int main()

{
    int n, e, i, j;

    /* Function declarations inside main (as asked) */

    int find(int);

    void unionSet(int, int);

    struct edge
    {
        int u, v, w;

        } edges[20], temp;

    printf("Enter number of vertices: ");

    scanf("%d", &n);

    printf("Enter number of edges: ");

    scanf("%d", &e);

    printf("Enter edges (u v weight):\n");

    for(i = 0; i < e; i++)

        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

    /* Initialize parent array */

    for(i = 0; i < n; i++)

        parent[i] = i;

    /* Sort edges by weight (Bubble sort) */

    for(i = 0; i < e - 1; i++)

    {

```



```

for(j = 0; j < e - i - 1; j++)
{
    if(edges[j].w > edges[j + 1].w)
    {
        temp = edges[j];
        edges[j] = edges[j + 1];
        edges[j + 1] = temp;
    }
}

int count = 0, cost = 0;

printf("\nEdges in Minimum Cost Spanning Tree:\n");

for(i = 0; i < e && count < n - 1; i++)
{
    int u = find(edges[i].u);
    int v = find(edges[i].v);
    if(u != v)
    {
        printf("%d -- %d = %d\n", edges[i].u, edges[i].v, edges[i].w);
        cost += edges[i].w;
        unionSet(u, v);
        count++;
    }
}

```

```
printf("\nMinimum Cost = %d\n", cost);  
return 0;  
}
```

Output:

Enter number of vertices: 4

Enter number of edges: 3

Enter edges (u v weight):

3 4

1 4

1 2

4 6

5 7

Edges in Minimum Cost Spanning Tree:

3 -- 4 = 1

4 -- 1 = 2

Minimum Cost = 3

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:	Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Roll No: Batch No: Experiment No: 06 Lab No: Aim : Write a program in C to find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.	

Program Code:

```
#include <stdio.h>

#define INF 999

int cost[10][10], visited[10];

/* Function definition outside main */

void prim(int n)
{
    int i, j;

    int mincost = 0;

    int edgeCount = 0;

    int u, v, min;

    visited[0] = 1; // Start from vertex 0

    printf("\nEdges in Minimum Cost Spanning Tree:\n");

    while(edgeCount < n - 1)
    {
```

```

min = INF;
    for(i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(j = 0; j < n; j++)
            {
                if(!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }
    printf("%d -- %d = %d\n", u, v, min);
    mincost += min;
    visited[v] = 1;
    edgeCount++;
}
printf("\nMinimum Cost = %d\n", mincost);
}

int main()

```

```

{
    int n, i, j;
/* Function declaration inside main */

    void prim(int);

    printf("Enter number of vertices: ");

    scanf("%d", &n);

    printf("Enter cost adjacency matrix (enter 0 if no edge):\n");

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }

        visited[i] = 0;
    }

    prim(n);

    return 0;
}

```

Output: Enter number of vertices: 4

Enter cost adjacency matrix (enter 0 if no edge):

0 10 6 5

10 0 0 15

6 0 0 4

5 15 4 0

Edges in Minimum Cost Spanning Tree:

0 -- 3 = 5

3 -- 2 = 4

0 -- 1 = 10

Minimum Cost = 19

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 07	Roll No: Lab No:	Batch No:
Aim : Write a program in C to from a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.		

Program Code:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, source;
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
```

```
    int graph[n][n];
```

```
    int dist[n];
```

```
    int visited[n];
```

```
printf("Enter adjacency matrix (enter 0 if no edge):\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    for(j = 0; j < n; j++)
```

```
    {
```

```
        scanf("%d", &graph[i][j]);
```

```
    }
```

```
}
```

```
printf("Enter source vertex (0 to %d): ", n-1);
```

```
scanf("%d", &source);
```

```
/* Initialization */
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    dist[i] = 9999;    // infinity
```

```
    visited[i] = 0;    // not visited
```

```
}
```

```
dist[source] = 0;
```

```
/* Dijkstra Algorithm */
```

```
for(i = 0; i < n-1; i++)
```

```
{
```



```
int min = 9999;
```

```
int u = -1;
```

```
for(j = 0; j < n; j++)
```

```
{
```

```
    if(visited[j] == 0 && dist[j] < min)
```

```
    {
```

```
        min = dist[j];
```

```
        u = j;
```

```
    }
```

```
}
```

```
visited[u] = 1;
```

```
for(j = 0; j < n; j++)
```

```
{
```

```
    if(visited[j] == 0 && graph[u][j] != 0)
```

```
    {
```

```
        if(dist[u] + graph[u][j] < dist[j])
```

```
        {
```

```
            dist[j] = dist[u] + graph[u][j];
```

```
        }
```

```
    }
```

```
}
```

```

    }

    /* Output */
    printf("\nShortest distances from source %d:\n", source);
    for(i = 0; i < n; i++)
    {
        printf("To vertex %d = %d\n", i, dist[i]);
    }

    return 0;
}

```

Output:

Enter number of vertices: 4

Enter adjacency matrix (enter 0 if no edge):

0 2 3 4

3 0 5 6

2 3 0 7

2 7 8 0

Enter source vertex (0 to 3): 2

Shortest distances from source 2:

To vertex 0 = 2

To vertex 1 = 3

To vertex 2 = 0

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 08 Aim : Write a program in C to implement Knapsack problems using Greedy method..	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

/* Function definition outside main */

float knapsack(int n, float weight[], float profit[], float capacity);

int main()
{
    int n, i;

    float weight[10], profit[10], capacity, maxProfit;

    /* Function declaration inside main */

    float knapsack(int, float[], float[], float);

    printf("Enter number of items: ");

    scanf("%d", &n);
```

```

for(i = 0; i < n; i++)
{
    printf("Enter weight and profit of item %d: ", i + 1);
    scanf("%f %f", &weight[i], &profit[i]);
}

printf("Enter capacity of knapsack: ");
scanf("%f", &capacity);

maxProfit = knapsack(n, weight, profit, capacity);

printf("Maximum profit = %.2f\n", maxProfit);

return 0;
}

/* Function to calculate maximum profit */
float knapsack(int n, float weight[], float profit[], float capacity)
{
    float ratio[10], temp;
    int i, j;
    float totalProfit = 0.0;

    /* Calculate profit/weight ratio */

```

```

for(i = 0; i < n; i++)

    ratio[i] = profit[i] / weight[i];


/* Sort items based on ratio (descending order) */
for(i = 0; i < n; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(ratio[i] < ratio[j])
        {
            temp = ratio[i];
            ratio[i] = ratio[j];
            ratio[j] = temp;


            temp = weight[i];
            weight[i] = weight[j];
            weight[j] = temp;


            temp = profit[i];
            profit[i] = profit[j];
            profit[j] = temp;
        }
    }
}

```

```
/* Select items greedily */
for(i = 0; i < n; i++)
{
    if(weight[i] <= capacity)
    {
        totalProfit += profit[i];
        capacity -= weight[i];
    }
    else
    {
        totalProfit += profit[i] * (capacity / weight[i]);
        break;
    }
}
return totalProfit;
}
```

Output:

Enter number of items: 3

Enter weight and profit of item 1: 60 10

Enter weight and profit of item 2: 100 20

Enter weight and profit of item 3: 120 30

Enter capacity of knapsack: 50

Maximum profit = 12.50

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 09	Roll No: Lab No:	Batch No:
Aim : Write a program in C to implement optimal binary search tree and also calculate the best case and worst case complexity.		

Program Code:

```
#include <stdio.h>

#define MAX 20
#define INF 9999

int cost[MAX][MAX], weight[MAX][MAX], root[MAX][MAX];
int keys[MAX], freq[MAX];

int main()
{
    int n, i, j, k, l;

    printf("Enter number of keys: ");
```

```
scanf("%d",&n);
```

```
printf("Enter keys:\n");
```

```
for(i=1;i<=n;i++)
```

```
    scanf("%d",&keys[i]);
```

```
printf("Enter frequencies:\n");
```

```
for(i=1;i<=n;i++)
```

```
    scanf("%d",&freq[i]);
```

```
// Initialize matrices
```

```
for(i=0;i<=n;i++)
```

```
{
```

```
    cost[i][i] = 0;
```

```
    weight[i][i] = 0;
```

```
}
```

```
// Calculate weights and costs
```

```
for(l=1;l<=n;l++)
```

```
{
```

```
    for(i=0;i<=n-l;i++)
```

```
    {
```

```
        j = i+l;
```

```
        weight[i][j] = weight[i][j-1] + freq[j];
```



```
cost[i][j] = INF;
```

```
for(k=i+1;k<=j;k++)
```

```
{
```

```
    int c = cost[i][k-1] + cost[k][j];
```

```
    if(c < cost[i][j])
```

```
    {
```

```
        cost[i][j] = c;
```

```
        root[i][j] = k;
```

```
    }
```

```
}
```

```
cost[i][j] += weight[i][j];
```

```
}
```

```
}
```

```
printf("\nCost Matrix:\n");
```

```
for(i=0;i<=n;i++)
```

```
{
```

```
    for(j=0;j<=n;j++)
```

```
        printf("%4d ",cost[i][j]);
```

```
    printf("\n");
```

```
}
```

```

printf("\nRoot Matrix:\n");
for(i=0;i<n;i++)
{
    for(j=1;j<=n;j++)
        printf("%4d ",root[i][j]);
    printf("\n");
}

printf("\nOptimal Binary Search Tree Cost = %d",cost[0][n]);
printf("\nRoot of OBST = %d\n",keys[root[0][n]]);

return 0;
}

```

Output:

Enter number of keys: 3

Enter keys:

10 20 30

Enter frequencies:

34 8 50

Cost Matrix:

0 34 50 142

0 0 8 66

0 0 0 50

0 0 0 0

Root Matrix:

1 1 3

0 2 3

0 0 3

Optimal Binary Search Tree Cost = 142

Root of OBST = 30

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 10	Roll No: Lab No:	Batch No:
Aim : Write a program in C to implement huffman Code using greedy methods and also calculate the best case and worst case complexity.		

Program Code:

```
#include <stdio.h>

#include <stdlib.h>

struct Node
{
    char data;          // Character
    int freq;           // Frequency of character
    struct Node *left;  // Left child
    struct Node *right; // Right child
};

int main()
{
    // Function declarations inside main
```

```
struct Node* createNode(char data, int freq);  
void printCodes(struct Node* root, int arr[], int top);
```

```
int n, i;
```

```
printf("Enter number of characters: ");  
scanf("%d", &n);
```

```
char ch[n];  
int freq[n];
```

```
for(i = 0; i < n; i++)  
{  
    printf("Enter character and frequency: ");  
    scanf(" %c %d", &ch[i], &freq[i]);  
}
```

```
struct Node* nodes[n];
```

```
// Create initial nodes
```

```
for(i = 0; i < n; i++)  
    nodes[i] = createNode(ch[i], freq[i]);
```

```
int size = n;
```

```
// Build Huffman Tree
```

```
while(size > 1)
```

```
{
```

```
    int min1 = 0, min2 = 1;
```

```
    // Find two smallest frequencies
```

```
    for(i = 0; i < size; i++)
```

```
    {
```

```
        if(nodes[i]->freq < nodes[min1]->freq)
```

```
            min1 = i;
```

```
    }
```

```
    for(i = 0; i < size; i++)
```

```
    {
```

```
        if(i != min1 && nodes[i]->freq < nodes[min2]->freq)
```

```
            min2 = i;
```

```
    }
```

```
    // Create new internal node
```

```
    struct Node* newNode = createNode('$',
```

```
        nodes[min1]->freq + nodes[min2]->freq);
```

```
    newNode->left = nodes[min1];
```

```

newNode->right = nodes[min2];

// Replace min1 with new node
nodes[min1] = newNode;

// Remove min2
nodes[min2] = nodes[size - 1];
size--;
}

int arr[100], top = 0;

printf("\nHuffman Codes:\n");
printCodes(nodes[0], arr, top);

return 0;
}

// Function to create node
struct Node* createNode(char data, int freq)
{
    struct Node* temp =
        (struct Node*)malloc(sizeof(struct Node));

```

```

temp->data = data;
temp->freq = freq;
temp->left = temp->right = NULL;

return temp;
}

// Function to print codes
void printCodes(struct Node* root, int arr[], int top)
{
    if(root->left)
    {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if(root->right)
    {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if(root->left == NULL && root->right == NULL)
    {

```



```
    printf("%c: ", root->data);  
    for(int i = 0; i < top; i++)  
        printf("%d", arr[i]);  
    printf("\n");  
}  
}
```

Output:

Enter number of characters: 5

Enter character and frequency: A 5

Enter character and frequency: B 9

Enter character and frequency: C 12

Enter character and frequency: D 13

Enter character and frequency: E 16

Huffman Codes:

A: 00

B: 01

E: 1

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 11 Aim : Write a program in C to find Minimum number of multiplications in Matrix Chain Multiplication/.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>
#include <limits.h>
```

```
int main()
{
    int n, i, j, k, len;

    printf("Enter number of matrices: ");
    scanf("%d", &n);

    int p[n + 1];

    printf("Enter dimensions array:\n");
    for(i = 0; i <= n; i++)
```

```
scanf("%d", &p[i]);
```

```
int m[n + 1][n + 1];
```

```
// cost is zero when multiplying one matrix
```

```
for(i = 1; i <= n; i++)
```

```
    m[i][i] = 0;
```

```
// length of chain
```

```
for(len = 2; len <= n; len++)
```

```
{
```

```
    for(i = 1; i <= n - len + 1; i++)
```

```
    {
```

```
        j = i + len - 1;
```

```
        m[i][j] = INT_MAX;
```

```
        for(k = i; k < j; k++)
```

```
        {
```

```
            int cost = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
```

```
            if(cost < m[i][j])
```

```
                m[i][j] = cost;
```

```
        }
```

```
    }
```

```
}

printf("Minimum number of multiplications = %d\n", m[1][n]);

return 0;

}
```

Output:

Enter number of matrices: 4

Enter dimensions array:

3 5 6 7 8

Minimum number of multiplications = 384

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 12 Aim : Write a Program in C to find only length of Longest Common Subsequence.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#include <string.h>

int main()
{
    char X[100], Y[100];

    int m, n;

    int L[100][100];

    int i, j;

    // Input two strings

    printf("Enter first string: ");

    scanf("%s", X);
```

```
printf("Enter second string: ");
```

```
scanf("%s", Y);
```

```
// Find length of strings
```

```
m = strlen(X);
```

```
n = strlen(Y);
```

```
// Step 1: Initialize first row and first column as 0
```

```
for(i = 0; i <= m; i++)
```

```
    L[i][0] = 0;
```

```
for(j = 0; j <= n; j++)
```

```
    L[0][j] = 0;
```

```
// Step 2: Fill the table using LCS formula
```

```
for(i = 1; i <= m; i++)
```

```
{
```

```
    for(j = 1; j <= n; j++)
```

```
    {
```

```
        if(X[i-1] == Y[j-1])
```

```
            L[i][j] = 1 + L[i-1][j-1];
```

```
        else
```

```
        {
```

```
            if(L[i-1][j] > L[i][j-1])
```

```
        L[i][j] = L[i-1][j];
    else
        L[i][j] = L[i][j-1];
    }
}

// Final Answer

printf("Length of Longest Common Subsequence = %d", L[m][n]);

return 0;
}
```

Output:

Enter first string: ABCDGH

Enter second string: EFGHIJ

Length of Longest Common Subsequence = 2

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 13 Aim : Write programs in C to implement DFS and BFS. Compare the time complexity.	Roll No: Lab No:	Batch No:

Program code:BFS

```
#include <stdio.h>

#define MAX 10

int adj[MAX][MAX];
int visited[MAX];
int queue[MAX];
int n;

void bfs(int start)
{
    int front = -1, rear = -1;
    int i;
    queue[++rear] = start; // Insert start vertex
    visited[start] = 1;
    while(front != rear)
```



```

{
    int v = queue[++front]; // Remove from queue
    printf("%d ", v);
    for(i = 0; i < n; i++)
    {
        if(adj[v][i] == 1 && visited[i] == 0)
        {
            queue[++rear] = i; // Insert into queue
            visited[i] = 1;
        }
    }
}
}

```

```

int main()
{
    int i, j, start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++)
    {

```

```

        for(j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
        }
    }

    for(i = 0; i < n; i++)
        visited[i] = 0;

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    printf("BFS Traversal: ");
    bfs(start);

    return 0;
}

```

Output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 0 1

1 0 1 1

1 1 0 0

1 0 1 0

Enter starting vertex: 3

BFS Traversal: 3 0 2 1

Program Code DFS:

```
#include <stdio.h>

#define MAX 10

int adj[MAX][MAX];    // Adjacency matrix
int visited[MAX];      // To track visited vertices
int n;                // Number of vertices

// DFS function
void dfs(int v)
{
    int i;

    printf("%d ", v);    // Print current vertex
    visited[v] = 1;      // Mark as visited

    for(i = 0; i < n; i++) // Check all adjacent vertices
    {
        if(adj[v][i] == 1 && visited[i] == 0)
        {
            dfs(i);      // Recursive call
        }
    }
}
```

```
    }  
}  
}
```

```
int main()
```

```
{
```

```
    int i, j, start;
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter adjacency matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            scanf("%d", &adj[i][j]);
```

```
        }
```

```
    }
```

```
    for(i = 0; i < n; i++)
```

```
        visited[i] = 0;    // Initially all unvisited
```

```
    printf("Enter starting vertex: ");
```

```
scanf("%d", &start);

printf("DFS Traversal: ");
dfs(start);

return 0;
}
```

Output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 0 1

0 0 1 1

1 0 0 1

1 1 1 0

Enter starting vertex: 0

BFS Traversal: 0 1 3 2

<i>Vidya Pratishthan's</i> Arts, Science and Commerce College <i>Vidyanagari, Baramati</i> Department of Computer Science		Performed
Name:		Submitted
Class: F.Y. M.Sc. (Comp. Sci.) Roll No:		Remark & Signature
Experiment No: 14 Lab No:		
Aim : Write a program in C for finding Topological sorting for Directed Acyclic Graph (DAG).		

Program Code:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 10

int adj[MAX][MAX];    // Adjacency matrix
int visited[MAX];      // To mark visited vertices
int stack[MAX];        // Stack to store result
int top = -1;          // Stack pointer
int n;                 // Number of vertices

// DFS Function
void dfs(int v)
```

```

{
    visited[v] = 1; // Mark current vertex as visited

    for(int i = 0; i < n; i++)
    {
        if(adj[v][i] == 1 && visited[i] == 0)
        {
            dfs(i); // Recursive call
        }
    }

    stack[++top] = v; // Push vertex to stack after visiting all neighbors
}

```

```

int main()
{
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++)
    {

```

```

    for(j = 0; j < n; j++)
    {
        scanf("%d", &adj[i][j]);
    }
}

// Initialize visited array
for(i = 0; i < n; i++)
{
    visited[i] = 0;
}

// Call DFS for each unvisited vertex
for(i = 0; i < n; i++)
{
    if(visited[i] == 0)
    {
        dfs(i);
    }
}

printf("Topological Sorting Order:\n");
while(top != -1)
{

```



```
    printf("%d ", stack[top--]);  
}
```

```
    return 0;  
}
```

Output:

Enter number of vertices: 2

Enter adjacency matrix:

1 4

2 6

Topological Sorting Order:

1 0

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 15 Aim : Write a program in C to determine if a given graph is a Hamiltonian cycle or not.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#define V 5

int path[V];

int isSafe(int v, int graph[V][V], int pos)
{
    int i;

    if(graph[path[pos-1]][v] == 0)
        return 0;

    for(i = 0; i < pos; i++)
        if(path[i] == v)
```

```
    return 0;
```

```
    return 1;
```

```
}
```

```
int hamCycleUtil(int graph[V][V], int pos)
```

```
{
```

```
    int v;
```

```
    if(pos == V)
```

```
    {
```

```
        if(graph[path[pos-1]][path[0]] == 1)
```

```
            return 1;
```

```
        else
```

```
            return 0;
```

```
    }
```

```
    for(v = 1; v < V; v++)
```

```
    {
```

```
        if(isSafe(v, graph, pos))
```

```
        {
```

```
            path[pos] = v;
```

```
            if(hamCycleUtil(graph, pos+1) == 1)
```

```
                return 1;
```

```
            path[pos] = -1;
```

```

    }
}

return 0;
}

int hamCycle(int graph[V][V])
{
    int i;

    for(i = 0; i < V; i++)
        path[i] = -1;

    path[0] = 0;

    if(hamCycleUtil(graph, 1) == 0)
    {
        printf("Solution does not exist");
        return 0;
    }

    printf("Hamiltonian Cycle:\n");
    for(i = 0; i < V; i++)
        printf("%d ", path[i]);

```

```
    printf("%d", path[0]);

    return 1;
}
```

```
int main()
{
    int graph[V][V] = {
        {0,1,0,1,0},
        {1,0,1,1,1},
        {0,1,0,0,1},
        {1,1,0,0,1},
        {0,1,1,1,0}
    };
    hamCycle(graph);

    return 0;
}
```

Output:

Hamiltonian Cycle:

0 1 2 4 3 0

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 16 Aim : Write a C Program in C to implement Traveling Salesman Problem using nearest neighbor algorithm.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

int main()
{
    int n, i, j;

    // Function declaration inside main()
    void nearestNeighbor(int n, int cost[10][10]);

    int cost[10][10];

    printf("Enter number of cities: ");
    scanf("%d", &n);
```

```

printf("Enter cost matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        scanf("%d", &cost[i][j]);
    }
}

nearestNeighbor(n, cost);

return 0;
}

// Function definition
void nearestNeighbor(int n, int cost[10][10])
{
    int visited[10] = {0};
    int i, city = 0, count = 1;
    int min, nextCity;
    int totalCost = 0;

    printf("\nTour Path: ");

```

```
visited[city] = 1;
```

```
printf("%d ", city);
```

```
while(count < n)
```

```
{
```

```
    min = 999;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(cost[city][i] != 0 && visited[i] == 0)
```

```
        {
```

```
            if(cost[city][i] < min)
```

```
            {
```

```
                min = cost[city][i];
```

```
                nextCity = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    visited[nextCity] = 1;
```

```
    printf("-> %d ", nextCity);
```

```
    totalCost += min;
```

```
    city = nextCity;
```



```
        count++;  
    }  
  
    totalCost += cost[city][0];  
    printf("-> 0");  
  
    printf("\nTotal Cost: %d\n", totalCost);  
}
```

Output:

Enter number of cities: 4

Enter cost matrix:

30 47 13 25

45 68 84 61

23 64 43 12

98 14 21 76

Tour Path: 0 -> 2 -> 3 -> 1 -> 0

Total Cost: 84

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 17 Aim : Write a program in C a to implement Graph Coloring Algorithm..	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int color[MAX];

int n, m; // n = number of vertices, m = number of colors

// Function to check if it's safe to assign color
int isSafe(int v, int c) {
    for (int i = 0; i < n; i++) {
        if (graph[v][i] == 1 && color[i] == c)
            return 0;
    }
    return 1;
}
```

```

// Backtracking function
int graphColoring(int v) {
    if (v == n)
        return 1;
    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;
            if (graphColoring(v + 1))
                return 1;
            color[v] = 0; // Backtrack
        }
    }
    return 0;
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
}

```

```

    }

    printf("Enter number of colors: ");
    scanf("%d", &m);
    for (int i = 0; i < n; i++)
        color[i] = 0;

    if (graphColoring(0)) {
        printf("\nSolution exists:\n");
        for (int i = 0; i < n; i++) {
            printf("Vertex %d ---> Color %d\n", i, color[i]);
        }
    } else {
        printf("\nNo solution exists.");
    }

    return 0;
}

```

Output:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 18 Aim : Write a program in C to implement Sum of Subset by Backtracking.	Roll No: Lab No:	Batch No:

Program Code:

```
#include<stdio.h>

#define MAX 10

void subset(int s[], int x[], int n, int sum, int k, int target);
void print(int x[], int s[], int n);
int main()
{
    // Function declarations inside main (as required)
    void subset(int s[], int x[], int n, int sum, int k, int target);
    void print(int x[], int s[], int n);

    int s[MAX], x[MAX];

    int n, target, i;
```

```
printf("Enter number of elements: ");
```

```
scanf("%d",&n);
```

```
printf("Enter elements:\n");
```

```
for(i=0;i<n;i++)
```

```
    scanf("%d",&s[i]);
```

```
printf("Enter target sum: ");
```

```
scanf("%d",&target);
```

```
subset(s,x,n,0,0,target);
```

```
return 0;
```

```
}
```

```
void subset(int s[], int x[], int n, int sum, int k, int target)
```

```
{
```

```
    int i;
```

```
    if(sum == target)
```

```
    {
```

```
        print(x,s,n);
```

```
        return;
```

```
    }
```

```
    for(i=k;i<n;i++)
```

```
    {
```

```
        x[i] = 1;
```

```
        if(sum + s[i] <= target)
```

```

        subset(s,x,n,sum+s[i],i+1,target);
    x[i] = 0;
}
}
void print(int x[], int s[], int n)
{
    int i;
    printf("Subset: ");
    for(i=0;i<n;i++)
    {
        if(x[i] == 1)
            printf("%d ",s[i]);
    }
    printf("\n");
}

```

Output:

Enter number of elements: 4

Enter elements:

2 4 6 8

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 19 Aim : Write a program in C to solve N Queens Problem using Backtracking.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#include <stdlib.h>

int x[20], count=0;

/* Check whether queen can be placed */

int place(int k,int i)
{
    int j;
    for(j=1;j<k;j++)
    {
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    }
    return 1;
}
```



```

}

/* N-Queen function */
void nqueen(int k,int n)
{
    int i,j;

    for(i=1;i<=n;i++)
    {
        if(place(k,i))
        {
            x[k]=i;

            if(k==n)
            {
                count++;
                printf("\nSolution %d:\n",count);

                for(i=1;i<=n;i++)
                {
                    for(j=1;j<=n;j++)
                    {
                        if(x[i]==j)
                            printf(" Q ");

```

```

        else
            printf(" . ");
        }
        printf("\n");
    }
}

else
    nqueen(k+1,n);
}
}
}

int main()
{
    int n;

    printf("Enter number of queens: ");
    scanf("%d",&n);

    nqueen(1,n);

    if(count==0)
        printf("No solution exists");

```

```
    return 0;  
}
```

Output:

Enter number of queens: 4

Solution 1:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Solution 2:

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

<i>Vidya Pratishthan's</i> Arts, Science and Commerce College <i>Vidyanagari, Baramati</i> Department of Computer Science		Performed
Name:		Submitted
Class: F.Y. M.Sc. (Comp. Sci.)		Remark & Signature
Roll No:	Batch No:	
Experiment No: 20	Lab No:	
Aim : Write a program in C to solve 4 Queens Problem using Backtracking		

Program Code:

```
#include <stdio.h>

#include <stdlib.h>

int x[10]; // Stores column positions

// Check if queen can be placed safely
int place(int k, int i) {
    for (int j = 1; j < k; j++) {
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}
```

```
// Print solution
```

```
void printSolution(int n) {  
    printf("\nSolution:\n");  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (x[i] == j)  
                printf(" Q ");  
            else  
                printf(" . ");  
        }  
        printf("\n");  
    }  
}
```

```
// Backtracking function
```

```
void nQueens(int k, int n) {  
    for (int i = 1; i <= n; i++) {  
        if (place(k, i)) {  
            x[k] = i;  
  
            if (k == n)  
                printSolution(n);  
            else
```

```

    nQueens(k + 1, n);
        }
    }
}

int main() {
    int n = 4; // Fixed for 4-Queens

    printf("Solutions for 4-Queens Problem:\n");
    nQueens(1, n);

    return 0;
}

```

Output:

Solutions for 4-Queens Problem:

Solution:

```

. Q . .
. . . Q
Q . . .
. . Q .

```

Solution:

```

. . Q .
Q . . .
. . . Q

```

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science		Performed
Name:		Submitted
Class: F.Y. M.Sc. (Comp. Sci.)		Remark & Signature
Experiment No: 21	Roll No:	Batch No:
Aim : Write a program in C to show board configuration of 4 queens problem		

Program Code:

```
#include <stdio.h>

#include <math.h>

int x[10]; // stores column position of queen in each row

int place(int k, int i)
{
    int j;
    for(j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0; // queen attacks
    }
    return 1; // safe position
}

void nqueen(int k, int n)
{

```

```

    int i, j;
    for(i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;

            if(k == n)
            {
                printf("\nSolution:\n");

                for(i = 1; i <= n; i++)
                {
                    for(j = 1; j <= n; j++)
                    {
                        if(j == x[i])
                            printf(" Q ");
                        else
                            printf(" . ");
                    }
                    printf("\n");
                }
            }
        }
    }
    else

```



```

        nqueen(k + 1, n);
    }
}
int main()
{
    int n = 4;
    nqueen(1, n);
    return 0;
}

```

Output:

Solution:

```

. Q . .
. . . Q
Q . . .
. . Q .

```

Solution:

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 22 Aim : Write a program in C to find out longest common subsequence from the given strings.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char X[100], Y[100];

    int m, n;

    printf("Enter first string: ");

    scanf("%s", X);

    printf("Enter second string: ");
```

```
scanf("%s", Y);
```

```
m = strlen(X);
```

```
n = strlen(Y);
```

```
int L[m+1][n+1];
```

```
// Build LCS table
```

```
for (int i = 0; i <= m; i++) {
```

```
    for (int j = 0; j <= n; j++) {
```

```
        if (i == 0 || j == 0)
```

```
            L[i][j] = 0;
```

```
        else if (X[i-1] == Y[j-1])
```

```
            L[i][j] = L[i-1][j-1] + 1;
```

```
        else
```

```
            L[i][j] = max(L[i-1][j], L[i][j-1]);
```

```
    }
```

```
}
```

```
// Length of LCS
```

```
int index = L[m][n];
```

```
char lcs[index + 1];
```

```
lcs[index] = '\0';
```

```

// Backtracking to find LCS string
int i = m, j = n;
while (i > 0 && j > 0) {
    if (X[i-1] == Y[j-1]) {
        lcs[index - 1] = X[i-1];
        i--;
        j--;
        index--;
    }
    else if (L[i-1][j] > L[i][j-1])
        i--;
    else
        j--;
}

printf("\nLongest Common Subsequence: %s\n", lcs);
printf("Length of LCS: %d\n", L[m][n]);

return 0;
}

```

Output:

Enter first string: ABCDGH

Enter second string: AEDFHR

Longest Common Subsequence: ADH

<i>Vidya Pratishthan's</i> Arts, Science and Commerce College <i>Vidyanagari, Baramati</i> Department of Computer Science		Performed Submitted Remark & Signature
Name:		
Class: F.Y. M.Sc. (Comp. Sci.)	Roll No:	Batch No:
Experiment No: 23	Lab No:	
Aim : Write a program in C to find out live node, E node and dead node from a given graph.		

Program Code:

```
#include <stdio.h>

#define MAX 5

int graph[MAX][MAX] = {
    {0,1,1,0,0},
    {1,0,0,1,0},
    {1,0,0,1,1},
    {0,1,1,0,1},
    {0,0,1,1,0}
};

int visited[MAX] = {0};

int main()
{
    int i, j;
```

```

int enode = 0;

printf("E-node : %d\n", enode);

visited[enode] = 1;

printf("Live Nodes : ");

for(i = 0; i < MAX; i++)
{
    if(graph[enode][i] == 1 && visited[i] == 0)
    {
        printf("%d ", i);
    }
}

printf("\nDead Nodes : ");

for(i = 0; i < MAX; i++)
{
    if(graph[enode][i] == 0 && i != enode)
    {
        printf("%d ", i);
    }
}return 0;
}

```

Output: E-node : 0

Live Nodes : 1 2

Dead Nodes : 3 4

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 24 Aim : Write a program in C to find out solution for travelling salesman problem using LCBB from a given matrix.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>

#define N 4

#define INF 999

int visited[N] = {0};

int mincost = INF;

void tsp(int cost[N][N], int city, int count, int curr_cost)
{
    int i;

    if(count == N && cost[city][0] > 0)
    {
        if(curr_cost + cost[city][0] < mincost)
            mincost = curr_cost + cost[city][0];

        return;
    }

    for(i = 0; i < N; i++)
```

```

    {
        if(visited[i] == 0 && cost[city][i] > 0)
        {
            visited[i] = 1;
            tsp(cost, i, count + 1, curr_cost + cost[city][i]);
            visited[i] = 0;
        }
    }
}

int main()
{
    int cost[N][N] = {
        {0,10,15,20},
        {10,0,35,25},
        {15,35,0,30},
        {20,25,30,0}
    };

    visited[0] = 1;

    tsp(cost, 0, 1, 0);

    printf("Minimum travelling cost = %d", mincost);

    return 0;
}

```

Output: Minimum travelling cost = 80

Vidya Pratishthan's Arts, Science and Commerce College Vidyanagari, Baramati Department of Computer Science Name:		Performed Submitted Remark & Signature
Class: F.Y. M.Sc. (Comp. Sci.) Experiment No: 25 Aim : Write a program in C to find out solution for 0/1 knapsack problem.	Roll No: Lab No:	Batch No:

Program Code:

```
#include <stdio.h>
```

```
int max(int a,int b)
```

```
{
```

```
    if(a>b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```
}
```

```
int main()
```

```
{
```

```
    int n, i, j, W;
```

```
printf("Enter number of items: ");
```

```
scanf("%d",&n);
```

```
int wt[n+1], val[n+1];
```

```
printf("Enter weights of items:\n");
```

```
for(i=1;i<=n;i++)
```

```
    scanf("%d",&wt[i]);
```

```
printf("Enter profits of items:\n");
```

```
for(i=1;i<=n;i++)
```

```
    scanf("%d",&val[i]);
```

```
printf("Enter knapsack capacity: ");
```

```
scanf("%d",&W);
```

```
int K[n+1][W+1];
```

```
for(i=0;i<=n;i++)
```

```
{
```

```
    for(j=0;j<=W;j++)
```

```
    {
```

```
        if(i==0 || j==0)
```

```
            K[i][j]=0;
```

```

        else if(wt[i] <= j)
            K[i][j]=max(val[i]+K[i-1][j-wt[i]], K[i-1][j]);
        else
            K[i][j]=K[i-1][j];
    }
}

printf("\nMaximum Profit = %d",K[n][W]);

return 0;
}

```

Output:

Enter number of items: 4

Enter weights of items:

2 5

4 8

Enter profits of items:

24

21

45

67

Enter knapsack capacity: 25

Maximum Profit = 157

